COMPUTER NETWORKS · DIGITAL ASSIGNMENT 2

NetSentry

On-Device Network Behaviour Monitor for Applications

Project documentation: architecture, pipeline, detection engine, implementation, testing and validation

8	2 s	54	0
explainable heuristics	end-to-end tick	automated tests	infrastructure needed

Nainika Pathak
Registration no. 24BCE0230

Version 0.1.0 · October 2026
Source: github.com/Nainikap/mycafee

A behaviour monitor, deliberately not an antivirus: metadata only, no payload decryption, every rule open and tunable.

Contents

1 Introduction . . . 4
1.1 What NetSentry is, and is not . . . 4
1.2 Objectives . . . 4
1.3 How this document is organised . . . 5
2 Background and Existing Systems . . . 6
2.1 The gap . . . 6
2.2 Novelty . . . 6
3 Scope . . . 7
3.1 Operating modes . . . 7
4 System Architecture . . . 8
4.1 Layers . . . 8
4.2 Components . . . 9
4.3 Runtime and threading model . . . 9
4.4 Key design decisions . . . 10
5 End-to-End Pipeline . . . 11
5.1 Step by step . . . 11
5.2 Slower loops . . . 12
5.3 Flow lifecycle . . . 12
5.4 Naming destinations . . . 12
5.5 Record and replay . . . 12
6 Capture Layer . . . 13
6.1 Socket table polling . . . 13
6.2 Process resolution and lineage . . . 13
6.3 Packet sniffer (full mode) . . . 13
6.4 TLS ClientHello, JA3 and JA3N . . . 14
6.5 DNS answers and user idle time . . . 14
7 Correlation Layer . . . 15
7.1 Attribution rules . . . 15
7.2 Flow direction . . . 15
7.3 Destination keys . . . 15
8 Detection Engine . . . 16
8.1 Features: buckets and windows . . . 16
8.2 Baselines . . . 16
8.3 The eight heuristics . . . 17
8.4 Scoring: noisy-OR . . . 18

8.5 Worked examples . . . 19
8.6 Alert lifecycle . . . 20
8.7 The feedback loop . . . 20
9 Data Model and Storage . . . 21
9.1 Write strategy and retention . . . 21
10 Technical Implementation . . . 22
10.1 Technology stack . . . 22
10.2 Repository layout . . . 22
10.3 Core interfaces . . . 22
10.4 Command-line interface . . . 23
10.5 The dashboard . . . 23
10.6 Configuration . . . 24
10.7 Engineering conventions . . . 24
11 Testing and Validation . . . 25
11.1 Automated test suite . . . 25
11.2 Synthetic scenario results . . . 25
11.3 Attack simulators . . . 26
11.4 Live measurements . . . 26
11.5 Bugs found by live testing . . . 26
11.6 Remaining validation work . . . 27
12 Limitations . . . 28
13 Future Work . . . 29
14 Running the Project . . . 30
14.1 Suggested demonstration (≈ 10 minutes) . . . 30
Appendix A: Default rules.yaml . . . 31
Appendix B: Glossary . . . 32
Appendix C: References . . . 33

1 Introduction

Every consumer machine runs a mix of applications that quietly open network connections: sync clients, telemetry, update checkers, and occasionally something less benign, such as spyware or a compromised binary beaconing to a command-and-control (C2) server. An ordinary user cannot tell **which process** is talking to **which host**, **how often** or **how much**. That gap is usually filled by heavyweight, closed-source security suites that rely on signature databases, cloud reputation and kernel drivers.

NetSentry is a lightweight, host-based network behaviour monitor that gives the user this visibility. It attributes every connection on the machine to the process that owns it, learns what is normal for each application, and scores deviations with **eight transparent, explainable heuristics**. The result is a live per-application severity, with plain-language evidence for every alert.

> **Design principle.** Every alert has to answer three questions: *which app*, *what did it do*, and *why is that unusual for this app*. A rule that cannot explain itself in one sentence with numbers does not belong in NetSentry.

1.1 What NetSentry is, and is not

NetSentry is…	NetSentry is not…
A behaviour monitor that sees **who** an app talks to, **how much** and **how often**	An antivirus: it has no malware signature database and does not scan files
Metadata-based: IPs, ports, timing, byte counts, DNS names, TLS fingerprints	A content inspector: it never decrypts TLS or reads payloads
Transparent: every weight and threshold is a line in a YAML file	A black-box ML model (an Isolation Forest is listed as future work)
A single-host daemon with zero infrastructure	A network sensor on a SPAN port, like Zeek or Suricata
Advisory: it scores and alerts; the user decides	A firewall: it does not block traffic

1.2 Objectives

The DA2 report set five objectives. The table shows where each one is met in the implementation.

#	Objective (from DA2)	Delivered by
1	Capture inbound/outbound connections and attribute each to its process (PID + executable)	`capture/socket_poller.py`, `process_resolver.py`, `pipeline/attributor.py`
2	Log metadata: destination IP/domain, port, protocol, volume and frequency over time	Flow table, enricher, SQLite `flows` and `app_minutes` tables
3	Rule-based heuristic engine that flags suspicious connections	Eight heuristics, noisy-OR scorer, alert manager (Section 8)
4	Dashboard/CLI with severity levels and per-app history	Textual TUI, `netsentry` CLI, per-minute history sparklines
5	Whitelist/blacklist of applications or destinations	Policy engine: allow / deny / mute / “this is normal”, from the TUI or CLI

1.3 How this document is organised

- **Sections 2–3**: background, the gap NetSentry fills, and its scope.
- **Sections 4–5**: system architecture and the end-to-end pipeline that runs every two seconds.
- **Sections 6–8**: each layer in depth: capture, correlation, and the detection engine (features, baselines, the eight heuristics, scoring, alerts and the feedback loop).
- **Sections 9–10**: data model, storage and the technical implementation (stack, modules, CLI, dashboard, configuration).
- **Section 11**: testing and validation, including real measurements and the bugs that live testing found.
- **Sections 12–14**: limitations, future work, and how to run the project.
- **Appendices**: the full default rule file, a glossary and references.

2 Background and Existing Systems

Five representative tools were reviewed, covering commercial per-app visibility, packet analysis and network intrusion detection. They split into two camps, and NetSentry is positioned in the gap between them.

System	Approach	Process attribution	Deployment	Limitation for an individual endpoint
GlassWire	Visual traffic history + firewall	Yes (Windows/Android)	Single host, closed source	Anomaly logic undocumented and not tunable
Little Snitch	Connection-time interception	Yes (macOS/Linux)	Single host, closed source	Allow/deny prompts, no behavioural risk ranking
Zeek	Scriptable behavioural analysis	No	Network chokepoint / sensor	Needs a SPAN port; no notion of a local process
Wireshark	Manual packet inspection	No (by default)	Single host, manual	Not a monitor: someone has to be watching
Suricata	Signature-based IDS/IPS	No	Network chokepoint / sensor	Catches known-bad patterns, not novel behaviour

2.1 The gap

The tools with per-process attribution (GlassWire, Little Snitch) are closed-source, tied to one platform, and offer no inspectable scoring logic. The tools with open, explainable, rule-based detection (Zeek, Suricata) are built to watch a network segment and have no concept of "which local process opened this". Wireshark is a manual tool and belongs to neither camp.

2.2 Novelty

	What NetSentry combines	Compared with
1	**Open, per-process attribution.** Process-level mapping, with the parent chain, where every heuristic and weight is a line in a config file that a grader can read and change.	GlassWire / Little Snitch (closed)
2	**Zero-infrastructure behavioural scoring.** A Zeek-style transparent rule engine running as a single-host daemon. A Suricata-style blocklist check is one heuristic among eight, not the whole strategy.	Zeek / Suricata (need a sensor)
3	**A feedback loop that learns the user's baseline.** Per-app baselines plus allow/deny/"this is normal" decisions correct the engine for each machine, instead of one fixed threshold for everyone.	All five (static rules or prompts)

3 Scope

NetSentry is deliberately scoped as a **network-behaviour monitor, not an antivirus**. This is what keeps it buildable and honest within a semester.

Component	Status	Notes
Connection capture (OS socket table)	In scope	psutil → GetExtendedTcpTable / GetExtendedUdpTable
Process attribution + lineage	In scope	PID, executable, user, command line, up to 4 parents
Packet capture for bytes, DNS, SNI	In scope	Optional; needs administrator rights + Npcap
JA3 / JA3N TLS fingerprinting	In scope	From the cleartext ClientHello; no decryption
Metadata heuristics + scoring	In scope	Eight heuristics, noisy-OR, per-app baselines
Payload / TLS decryption	Out of scope	Would need MITM certificate injection: invasive
Malware signature database	Out of scope	Not maintainable in a semester
Kernel driver / rootkit detection	Out of scope	Needs kernel-mode development and code signing
Blocking traffic	Out of scope (stretch)	Optional Windows Firewall rule, never automatic
IP reputation (AbuseIPDB)	Stretch (P8)	Config hooks exist; not implemented yet

> Metadata alone is genuinely useful. An unfamiliar background process uploading 200 MB to a new, unnamed IP at 3 AM while the user is away is a strong signal, without reading a single byte of payload. Section 8.5 shows that exact case scoring **77 (High)**.

3.1 Operating modes

Packet capture needs administrator rights, so NetSentry runs in one of two modes and degrades gracefully. Missing data is never treated as evidence of safety: the heuristics that need it report *n/a*.

Capability	Standard mode (no admin)	Full mode (admin + Npcap)
Socket table, PIDs, parent chain	Yes	Yes
Per-flow byte counts	—	Yes
Domain names	Reverse DNS only	DNS answers + TLS SNI + reverse DNS
TLS fingerprints (JA3/JA3N)	—	Yes
UDP flows (QUIC, DNS, VoIP)	—	Yes (attributed by local port)
Exact connection start times (SYN)	2 s poll resolution	Yes
Heuristics available	6 of 8 (H4 volume, H8 TLS are n/a)	All 8

4 System Architecture

NetSentry is a single Python process with four layers. Each layer can be tested in isolation, and data flows strictly left to right. The only arrow going backwards is the user's feedback.

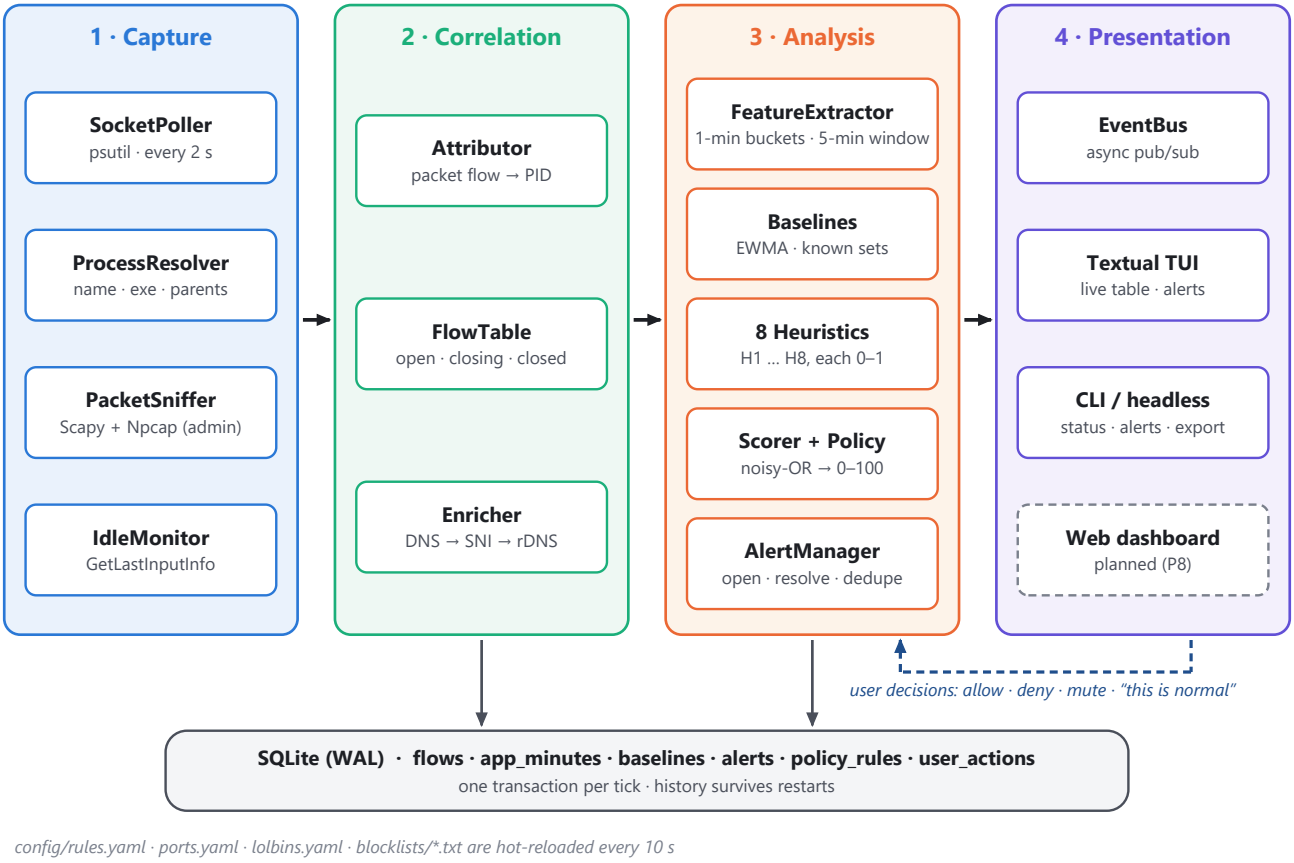


Figure 1. Layered architecture. Arrows show data flow; the dashed arrow is the feedback loop.

4.1 Layers

Layer	Responsibility	Input → Output	Tested in isolation by
1 · Capture	Read raw OS state and packets	OS → sockets, processes, packet stats, DNS answers, TLS hellos, idle time	`netsentry capture` vs `netstat -ano`
2 · Correlation	Turn samples into attributed, named flows	samples → Flow (PID, app, domain, bytes)	Hand-built ticks in `test_flows.py`
3 · Analysis	Features, baselines, heuristics, score, alerts	flows → Verdict per app, Alert	Synthetic windows in `test_heuristics.py`
4 · Presentation	Show state, take user decisions	events → UI; key presses → policy rules	Headless Textual pilot + synthetic scenario

The DA2 proposal described three layers. The middle **correlation** stage is now a layer of its own, because joining packets to sockets to processes is the trickiest part of the system and needs its own tests.

4.2 Components

Component	Module	What it does
SocketPoller	`capture/socket_poller.py`	Reads the connection table every 2 s; drops TIME_WAIT, PID-0 and loopback entries; separates listening sockets
ProcessResolver	`capture/process_resolver.py`	PID → name, exe, user, cmdline and parent chain; cached, evicted when the process exits
PacketSniffer	`capture/sniffer.py`	Scapy on Npcap: bytes/packets per 5-tuple, SYN times, DNS answers, TLS ClientHello
TLS parser	`capture/tls.py`	Hand-written ClientHello parser: SNI, JA3, JA3N
IdleMonitor	`capture/idle.py`	Seconds since last keyboard/mouse input (Win32 GetLastInputInfo)
Capture sources	`capture/source.py`	LiveSource, ReplaySource (JSONL), ListSource (tests), Recorder
Attributor	`pipeline/attributor.py`	Joins packet counters to sockets → PID; pending buffer → "unknown"
FlowTable	`pipeline/flow_table.py`	Flow lifecycle, direction (inbound/outbound), byte deltas
Enricher	`pipeline/enricher.py`	Domain for each IP: DNS cache → SNI → async reverse DNS; eTLD+1
Engine	`pipeline/engine.py`	Orchestrates one tick; reloads config; applies user actions
FeatureExtractor	`detection/features.py`	Per-app 1-minute buckets and the 5-minute window
Baseline	`detection/baseline.py`	EWMA statistics, known destinations/ports/JA3N, hour histogram
Heuristics	`detection/heuristics/*.py`	Eight pure functions → score 0–1 plus evidence
Scorer / Policy	`detection/scorer.py, policy.py`	Noisy-OR combination; allow/deny/mute overrides
AlertManager	`detection/alerts.py`	One alert per app per incident; resolve, reopen, false positive
Store	`storage/db.py, schema.sql`	SQLite in WAL mode; one transaction per tick
EventBus / UI	`bus.py, ui/tui/app.py, cli.py`	Async pub/sub; Textual dashboard; command line

4.3 Runtime and threading model

One asyncio event loop owns all engine state, so the engine needs no locks. Blocking OS calls run in a thread pool. The packet sniffer runs in Scapy's own thread and only writes to its private counter dictionary, which the engine swaps out atomically once per tick.

```
main thread ── asyncio event loop
   ├─ Engine.tick() every 2 s ............... orchestrates the pipeline (Section 5)
   │     ├─ run_in_executor(SocketPoller.poll)        blocking psutil call → thread
   │     ├─ run_in_executor(ProcessResolver.resolve_many)
   │     ├─ sniffer.drain()  → swap counters under a lock (microseconds)
   │     └─ attribute → flows → enrich → features → score → alerts → SQLite → EventBus
   ├─ Enricher reverse-DNS lookups ......... async, max 4 concurrent, 3 s timeout, cached 1 h
   └─ UI: Textual app (same loop) or headless console printer

sniffer thread ── Scapy AsyncSniffer callback
   └─ counters[5-tuple] += packet length; DNS answers / TLS hellos → lists (under a lock)
```

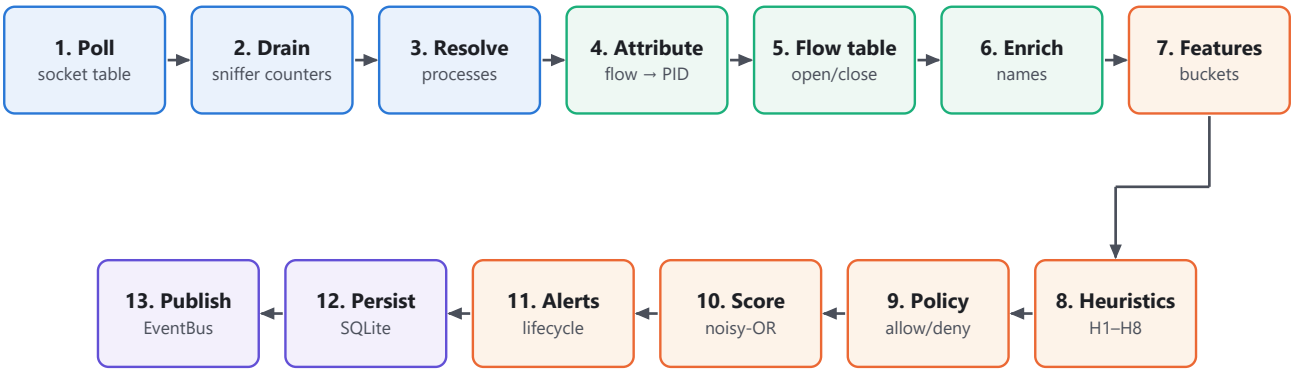
Every time-dependent component takes a `Clock`. Live runs use the system clock. Replays and tests inject a `FakeClock` driven by the recorded timestamps, which makes every run deterministic. The engine never imports a UI module; the TUI, the headless printer and a future web dashboard all subscribe to the same EventBus.

4.4 Key design decisions

#	Decision	Alternatives	Why
D1	Python 3.13 for the whole core	Node.js	psutil, Scapy and Textual cover all three layers in one language
D2	Poll the socket table + optional sniffer	ETW kernel provider; per-connection EStats	Works without admin; Npcap was already installed; ETW is complex to consume from Python
D3	1-minute buckets + EWMA baselines	Raw event windows; full recompute	Constant memory per app, easy to explain and persist
D4	Noisy-OR score combination	Weighted average	A weighted average dilutes one strong signal among quiet ones; noisy-OR lets signals reinforce
D5	SQLite in WAL mode	JSON files	No server, queryable history, readers never block the writer
D6	Textual TUI first, web later	Web first	Fastest path to an end-to-end demo; same EventBus
D7	Record / replay of raw ticks	Live-only testing	Deterministic tests, safe offline demo, offline tuning
D8	Normalised JA3 (JA3N) alongside JA3	JA4	Chrome randomises extension order; sorting keeps JA3 stable
D9	App identity = lower-cased exe path	Process name	Any binary can be called chrome.exe; the path is the real identity

5 End-to-End Pipeline

The engine runs one **tick** every two seconds. A tick is the unit of work for the whole system: it takes a snapshot of the machine and carries it all the way to the screen and the database.



one tick every 2 s · typical cost 5–15 ms on the test laptop · target under 250 ms

Figure 2. The 13 steps of a tick, coloured by layer (blue capture, green correlation, orange analysis, violet presentation).

5.1 Step by step

#	Stage	What happens
1	Poll	psutil.net_connections("inet") in a worker thread. TIME_WAIT/PID-0 entries and loopback remotes are dropped; LISTEN and UDP sockets are kept separately.
2	Drain sniffer	Swap the sniffer's counter dict for an empty one; collect DNS answers and TLS hellos (full mode only).
3	Resolve processes	New PIDs get name, exe, user, cmdline and parent chain, using one parent-PID snapshot for the whole batch.
4	Attribute	TCP: exact 5-tuple match to a socket. UDP: match on local port. Unmatched packets wait two ticks, then go to the app “unknown”.
5	Flow table	New key → open a flow; seen again → update; missing two polls → close. Flows accepted on a listening port are marked inbound.
6	Enrich	Find a name for the remote IP (DNS cache → SNI → reverse DNS) and its destination key.
7	Features	Add each flow's activity to its app's current 1-minute bucket; mark the app dirty.
8	Heuristics	Run all eight heuristics on every dirty app's 5-minute window against its baseline.
9	Policy	Apply the user's allow/deny/mute rules for this app and its destinations.
10	Score	Noisy-OR combination → 0–100 → severity (None/Low/Medium/High).
11	Alerts	Open, update, escalate or resolve this app's alert.
12	Persist	One SQLite transaction: new and closed flows, byte updates every 30 s, alerts.
13	Publish	Publish a TickResult (app rows, alert events, stats) on the EventBus for the UI.

5.2 Slower loops

Loop	Period	Job
Bucket close	60 s	Freeze every app's current minute; write it to app_minutes; fold clean minutes into the baseline
Baseline flush	5 min	Write changed baselines to SQLite
Config reload	10 s	Re-read rules.yaml and the static lists if their modification time changed; reload policy rules if the table changed
Retention	1 h	Delete closed flows older than 7 days and minute history older than 30 days

5.3 Flow lifecycle

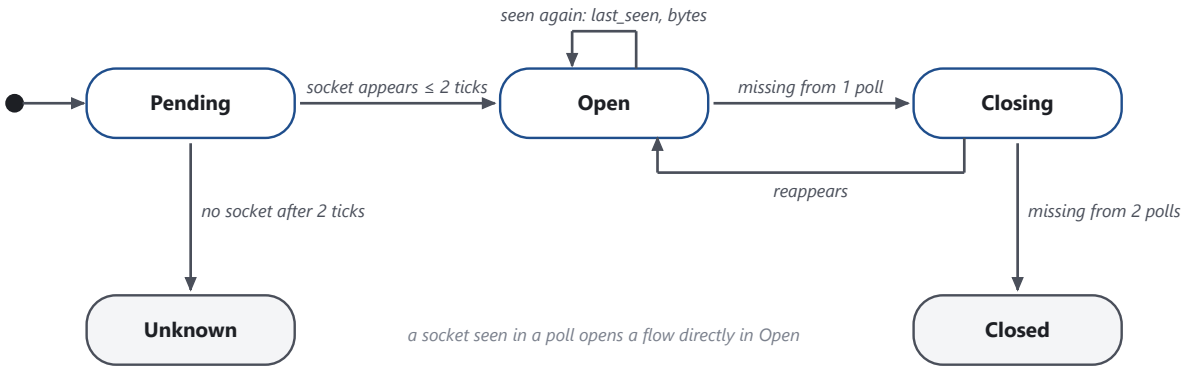


Figure 3. Flow states. A flow survives one missed poll, so a connection that briefly drops out of the table is not counted twice.

5.4 Naming destinations

A remote IP is turned into a **destination key**, which is what baselines remember:

- **1. DNS answer cache** (full mode): the sniffer parses DNS responses and maps each IP to the name that was originally asked for, following CNAME chains. Kept for max(TTL, 5 min).
- **2. TLS SNI** (full mode): the server name in that flow's ClientHello.
- **3. Reverse DNS** (both modes): async lookup, max 4 concurrent, 3 s timeout, cached for 1 hour.
- **Destination key**: the registrable domain (eTLD+1, e.g. `a1.cdn.example.co.uk` → `example.co.uk`) using an offline Public Suffix List. A bare IP falls back to its /24 (IPv4) or /48 (IPv6) prefix.
- A flow's destination is settled when a name is found, the lookup fails, or 4 s have passed. This stops one connection from being counted as two “new destinations” (once as an IP, then as a name).

5.5 Record and replay

`netsentry record` writes one JSON line per tick containing the **raw capture inputs**: sockets, processes, packet counters, DNS and TLS events, and idle time. `netsentry replay` feeds a recording through the identical pipeline with a fake clock. The same mechanism powers `netsentry demo`, a synthetic 25-minute scenario used for testing and presentations (Section 11.2).

6 Capture Layer

6.1 Socket table polling

On Windows, `psutil.net_connections(kind="inet")` wraps the IP Helper API calls `GetExtendedTcpTable` and `GetExtendedUdpTable`. These return every TCP/UDP endpoint with its owning PID, and need no administrator rights. A poll takes 3–30 ms on the test laptop.

- Entries in **TIME_WAIT** or **CLOSE**, or with PID 0, are skipped: Windows reports TIME_WAIT sockets under the idle process, so they have no attributable owner.
- **IPv4-mapped IPv6** addresses (`::ffff:1.2.3.4`) are unwrapped and IPv6 zone IDs stripped, so one host always has one spelling.
- Remote addresses classed as **loopback** are ignored by default. **LAN** means RFC 1918, CGNAT, link-local and ULA ranges, defined explicitly; Python's `is_private` also counts documentation ranges as private, which mislabelled test IPs as LAN during development.
- Sockets without a remote end (TCP LISTEN, all UDP) are kept as **listeners**. They are used to attribute UDP packets and to tell inbound connections from outbound ones.

6.2 Process resolution and lineage

For each PID the resolver records name, executable path, user, command line and the **parent chain** (up to four levels). The app identity is the lower-cased executable path, so a `svchost.exe` in `%TEMP%` is a different app from the real one in System32, and that difference is itself a signal (H7).

Performance work (measured on the development laptop, about 40 networked processes)

Version	Cold start	Steady state	Problem
First draft	3.6 s	~750 ms per tick	Re-validated every cached PID's create time on every call
Throttled re-check	3.5 s	spikes to 700 ms	Each Process.ppid() call on Windows snapshots all processes (~33 ms), once per parent hop
One ppid snapshot per batch	0.9 s	0–1 ms	—
Evict on exit instead of re-checking	0.9 s	0–1 ms, no spikes	—

A PID cannot be reused while its process still exists, so it is enough to evict cached PIDs that disappear from `psutil.pids()` (one cheap EnumProcesses call per poll). Parent-chain walks check that each parent is older than its child, which detects a parent PID that was reused by a younger process. Protected system processes that deny access appear as `pid:N` rather than crashing the poll.

6.3 Packet sniffer (full mode)

When NetSentry runs as administrator, a Scapy `AsyncSniffer` on Npcap captures with the BPF filter `ip or ip6` and `store=False`. The callback is kept short:

- Packet direction is decided by checking the source or destination against this machine's interface addresses (refreshed every 30 s). The key is always oriented from the local side (`proto, local ip, local port, remote ip, remote port`), which matches the socket table exactly.
- Bytes and packets are added per direction. An outgoing SYN without ACK records a precise **connection start time** (up to 20 per flow) for beacon detection.

- Payload is parsed only for **UDP source port 53** (DNS answers) and the **first three outgoing data packets** of a TCP flow (looking for a TLS ClientHello).
- If Scapy cannot open Npcap, NetSentry logs one clear reason and continues in standard mode.

6.4 TLS ClientHello, JA3 and JA3N

The ClientHello is the first message of a TLS handshake, and it is sent in clear text even in TLS 1.3. NetSentry parses it by hand (`capture/tls.py`) to extract the **SNI** (the server name) and a **JA3 fingerprint** of the client's TLS stack:

```
JA3 = MD5( version , ciphers , extensions , elliptic_curves , point_formats )
```

Each field is a dash-separated list of decimal values, for example `771,4865-4866-49195,0-23-65281-10-11,29-23-24,0`. GREASE values (0x0A0A, 0x1A1A, …), which browsers insert at random, are removed. Since 2023 Chrome also **shuffles its extension order** on every connection, which makes raw JA3 change every time. NetSentry therefore also computes **JA3N**, the same hash with the extension list sorted. JA3N stays stable for a given TLS library, so it is the value baselines remember. Modern ClientHellos with post-quantum key shares can exceed one TCP segment, so the sniffer reassembles up to three segments (16 KB maximum) before parsing.

6.5 DNS answers and user idle time

DNS responses are parsed with Scapy's DNS layer. Every A/AAAA record is mapped back to the *question* name, so `www.example.com → CNAME edge.cdn.net → 93.184.215.14` records the IP under `www.example.com`. User idle time comes from the Win32 call `GetLastInputInfo`, computed in 32-bit arithmetic so it stays correct when the tick counter wraps every 49.7 days.

7 Correlation Layer

7.1 Attribution rules

Case	Rule
TCP packet counters	Exact 5-tuple match against the socket table, or against an already-known flow
UDP packet counters	Match on local port against UDP listeners. Windows UDP sockets have no remote end, so this is the only way to attribute UDP
No match yet	Held in a *pending* buffer for 2 ticks; the socket often appears one poll after the first packet
Still no match	Attributed to the app `unknown`, so the traffic is counted and visible rather than silently dropped

7.2 Flow direction

A TCP flow whose local port is one of this machine's **LISTEN** ports was accepted from outside: this machine is the server, and the remote port is the client's random ephemeral port. Such flows are marked **inbound**. They still count towards destinations, but are excluded from the port rule (H1) and from beacon timing (H2). This rule was added after live testing showed a local test server being flagged for “connecting to non-standard port 5xxxx” (Section 11.5).

7.3 Destination keys

Grouping by registrable domain means content-delivery sub-domains do not each count as “new”, and a bare IP is grouped with its /24 neighbours (one cloud host often rotates within a block). The display keeps the full name. The key used for baselines is the coarser one.

8 Detection Engine

Every rule is a function of **features** (what the app is doing now) and its **baseline** (what it normally does). All numbers below are defaults from `config/rules.yaml` and can be changed while NetSentry is running.

8.1 Features: buckets and windows

Activity is rolled into a **per-app bucket for each minute**. The **window** a heuristic sees is the last five closed buckets plus the current partial one, so every decision is based on recent behaviour, not on a single poll.

Feature	Per bucket	Used by
`new_flows`	Outbound connections opened in the minute (pre-existing ones at startup are not counted)	H2 rate
`starts[ip:port]`	Last 20 connection start times per remote endpoint, kept across buckets	H2 beacon
`dests / new dests`	Destination keys contacted, and those not in the baseline	H3
`rports, listen ports`	Remote (protocol, port) pairs used; local LISTEN ports	H1
`bytes_out / bytes_in`	Summed per remote IP (full mode). Only outbound is scored, so downloads and game patches do not trigger it	H4
`ja3 / ja3n`	TLS fingerprints seen (full mode)	H8
`idle_max, hour`	User idle seconds; local hour of day	H6
`process`	Name, exe path, parent chain	H7

8.2 Baselines

Each app (keyed by exe path) has one baseline. It keeps an **exponentially weighted moving average (EWMA)** of the log of three per-minute features: `new_flows`, `unique_dests` and `bytes_out`. The log keeps heavy-tailed traffic manageable. The baseline also stores known destinations, ports and JA3N hashes, plus a 24-hour activity histogram.

```
v = log(1 + x)      mean ← mean + α·(v − mean)      var ← (1 − α)·(var + α·(v − mean_old)²)
```

With α = 0.02, the half-life is about 35 *active* minutes. The baseline is only updated on minutes in which the app actually did something, so an app that runs once an hour is compared with its own active minutes, not diluted by zeros. Deviation is measured as a z-score and mapped linearly to 0–1:

```
z = (v − mean) / max(√var, 0.5)          dev = clamp((z − 3) / (6 − 3), 0, 1)
```

Concept	Rule
App maturity	≥ 30 active minutes **and** ≥ 24 h since first seen (demo profile: 5 minutes and 10 minutes). Statistical rules only judge mature apps.
System maturity	NetSentry itself has run for the learning period. The start time is stored in the database, so it survives restarts.
New app	An app first seen *after* the system matured. Brand-new software appearing on an established machine is suspicious in itself, which closes the “new malware has no baseline” gap.
Poisoning guard	A minute only teaches the baseline if the app's verdict stayed below Medium during it, or the user later said “this is normal”. An attacker cannot slowly train the engine to accept an exfiltration.

8.3 The eight heuristics

Each heuristic is a pure function `(window, baseline, context) → (score 0–1 or n/a, evidence)`. Its score is the maximum over its sub-rules. Its **weight** is the most it can contribute on its own.

H1 · Unusual port usage weight 0.45

Sub-rule	Score
Remote port on the suspicious list: 4444 (Metasploit), 1337, 31337, 6666–6669 (IRC), 5555, 9001/9030 (Tor), 12345, 23	1.0
Mature app uses a **TCP** port outside the common list that it has never used before	0.5 (0.6 if ≥ 49152)
Mature app starts listening on a new local port	0.5

UDP is skipped for the "new port" rule because WebRTC, games and VoIP legitimately use random high UDP ports. Inbound flows are skipped entirely.

Evidence example: `connected to 198.51.100.23:4444 (suspicious port: Metasploit default)`

H2 · Connection frequency spike / beaconing weight 0.55

Sub-rule	Score
Rate: dev(new_flows) for a mature app, only if ≥ 10 new connections in the minute	0–1
Beacon: ≥ 6 connection starts to one ip:port in the last 5 min; coefficient of variation cv = std/mean of the gaps. cv ≤ 0.10 → 1.0, cv ≥ 0.35 → 0, linear between	0–1
Beacon to a destination the mature app already knows, or while NetSentry is still learning	× 0.3

Malware C2 implants "check in" at machine-regular intervals; humans do not. Legitimate heartbeats (Teams, Slack) are also regular, but go to known destinations, which is why they are damped. In standard mode only gaps ≥ 10 s are trusted, because start times are quantised to the 2 s poll.

Evidence example: `20 connections to 198.51.100.23:4444 every 30.1 s ± 1.0 s (cv 0.03)`

H3 · New / unrecognised destination weight 0.35

Sub-rule	Score
Mature app contacts destinations not in its baseline: 0.5 + 0.1 × (n − 1), capped at 0.8	0.5–0.8
New app (first seen after the system matured) uses the network	0.5
Bonus if any new destination has no name at all (direct-to-IP)	+ 0.2
All new destinations are on the LAN	× 0.5
System or app still learning	n/a

Evidence example: `first contact with 185.199.110.153:443; direct-to-IP, no DNS name`

H4 · Outbound data volume anomaly (full mode) weight 0.60

Sub-rule	Score
Mature app: dev(bytes_out per minute), only if ≥ 5 MB in the minute	0–1
New app: ≥ 50 MB in one minute	0.7

The floor stops a tiny app jumping from 1 KB to 50 KB from triggering it. Destinations the user allowed for this app are subtracted first.

Evidence example: `uploaded 74.5 MB in 1 min to 185.199.110.153; usually about 353.8 KB/min (z = 10.6)`

H5 · Known-bad IP / domain weight 0.95 + override

Sub-rule	Score
Remote IP in blocklists/ips.txt (single IPs or CIDR ranges)	1.0
Domain, or any parent domain, in blocklists/domains.txt	1.0

Any hit raises the final score to at least 80 (High). A user "allow" cannot hide it; it is capped at 60 and labelled allowed-but-blocklisted.

H6 · Off-hours activity weight 0.25 (amplifier)

Sub-rule	Score
User idle ≥ 10 min while the app is significantly active (≥ 3 new connections or ≥ 1 MB out)	0.6
Hour inside quiet hours (01:00–06:00), or a mature app is active in an hour holding < 2 % of its history	0.6
Both	1.0

Its weight is low on purpose. On its own it reaches Low at most, but it lifts a real anomaly a severity level.

H7 · Suspicious process lineage weight 0.60

Sub-rule	Score
Masquerading: name of a Windows system binary (svchost.exe, lsass.exe, …) running from outside the system folders	1.0
Office document / PDF reader / browser → script host (PowerShell, cmd, wscript, mshta, rundll32, certutil, …) that goes online	1.0
Any script host / LOLBin making a network connection	0.5
Executable runs from a staging folder (%TEMP%, Downloads, Users\Public)	0.6

Evidence example: `explorer.exe → WINWORD.EXE → powershell.exe: a document/browser process spawned a script host that went online`

H8 · TLS fingerprint (full mode) weight 0.35

Sub-rule	Score
JA3 or JA3N on the known-bad list (e.g. abuse.ch SSLBL)	1.0
Mature app presents a JA3N it has never used: its TLS client stack changed	0.5

This catches a process that normally uses Windows SChannel suddenly speaking with an embedded Go or Python TLS client. Known false positive: an app update that changes its TLS library, which is why the weight is low.

8.4 Scoring: noisy-OR

The available heuristic scores are combined as:

$$S = 1 - \prod_i (1 - w_i \cdot s_i) \qquad \text{score} = \text{round}(100 \cdot S)$$

This is the probability that at least one of several independent signals is "real", if signal i is real with probability $w_i \cdot s_i$. It has three properties that make it a good fit:

- **A strong single signal is not diluted.** With a plain weighted average across eight heuristics, a blocklist hit (1.0) next to seven zeros would score about 12. With noisy-OR it scores 95.
- **Independent weak signals reinforce each other.** A new destination (18) plus a volume spike (60) plus off-hours activity combine to 77.

- **It never exceeds 100 and never decreases when a signal is added.** Both properties are checked automatically with Hypothesis property-based tests.
- Heuristics that are unavailable (n/a) are simply left out of the product.

Score	Severity	Behaviour
0–24	None	Shown in the table, no colour
25–49	Low	Highlighted yellow, no alert
50–74	Medium	Alert opened, toast in the dashboard
75–100	High	Alert opened, red toast in the dashboard

Overrides, applied in this order

- 1. **Deny rule** for the app or one of its destinations → score at least 90.
- 2. **Blocklist hit** → score at least 80; if the app is allowed, capped at 60 instead.
- 3. **Allow rule** for the whole app → 0, shown as "allowed". Destination-level allows remove only that destination's traffic from H1–H4.
- 4. **Mute** → the score is computed and shown, but no alert is opened.

8.5 Worked examples

These rows are unit tests in `tests/unit/test_scorer.py`; the arithmetic is checked on every run.

Scenario	Signals (w · s)	Computation	Result
Background app uploads 200 MB to a new bare IP at 3 AM while the user sleeps	H4 0.60·1.0, H3 0.35·0.7, H6 0.25·1.0	1 − 0.40 · 0.755 · 0.75 = 0.77	**77 High**
C2 beacon every 30 s to a bare IP on port 4444	H2 0.55·1.0, H1 0.45·1.0, H3 0.35·0.7	1 − 0.45 · 0.55 · 0.755 = 0.81	**81 High**
Word spawns PowerShell, which downloads from a new domain	H7 0.60·1.0, H3 0.35·0.5	1 − 0.40 · 0.825 = 0.67	**67 Medium**
Chrome opens a site never visited before	H3 0.35·0.5	0.175	18 None
OneDrive uploads a large folder in the afternoon	H4 0.60·1.0	0.60	60 Medium (false positive → "this is normal")
Any app contacts a blocklisted IP	H5 0.95·1.0	0.95 (override ≥ 80)	**95 High**

8.6 Alert lifecycle

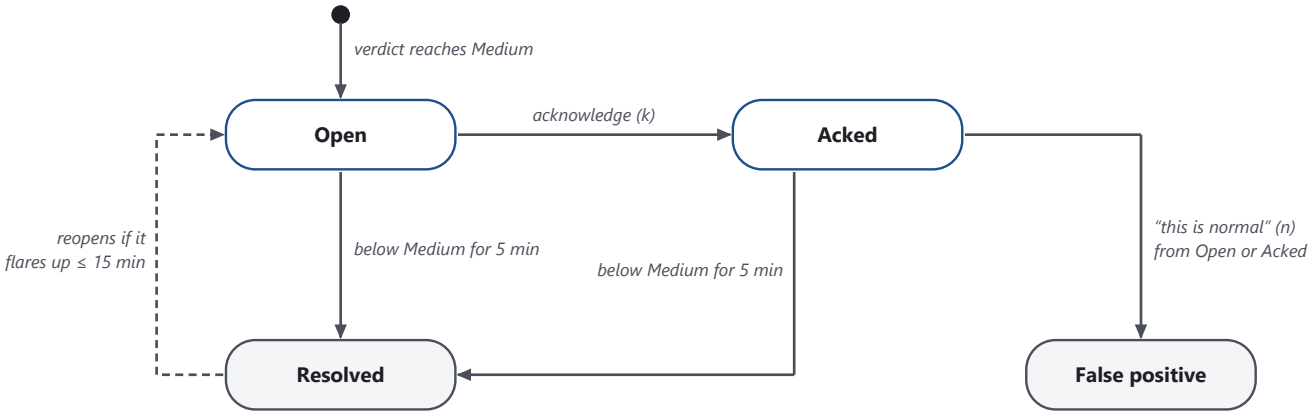


Figure 4. Alert states. There is one active alert per app per incident.

An alert opens when an app's verdict reaches Medium (configurable). While the app stays at that level, the same alert is updated in place: its peak score, top heuristic and evidence always describe the strongest moment, refreshed with the latest numbers at an equal score. The alert resolves after the app has stayed below the threshold for 5 minutes. If the same app flares up again within 15 minutes of resolving, the old alert is reopened rather than a new one created. An alert's summary is its top two evidence strings.

8.7 The feedback loop

User action	TUI key / CLI	Effect (applied on the next tick, ≤ 2 s)
Allow app	a / netsentry allow --app X	Policy rule; the app's score becomes 0 (blocklist still shows, capped at 60)
Allow destination for app	netsentry allow --app X --dest D	Destination excluded from H1–H4 for that app and added to its baseline
Deny app / destination	d / netsentry deny …	Score forced to at least 90 (High)
Mute	m / netsentry mute … --minutes N	Scored and shown, but no alerts (1 h by default)
This is normal	n	Alert closed as false positive; the alerted minutes are folded into the baseline. The only way flagged behaviour becomes “normal”.
Acknowledge	k	Alert marked as seen; it stays open until it resolves

Every decision is written to `user_actions` with a timestamp, so the alert history shows who changed what, and the validation can measure how quickly the false-positive rate falls.

9 Data Model and Storage

All state lives in one SQLite database at `%LOCALAPPDATA%\NetSentry\netsentry.db` (override with `--db`), in WAL mode with `synchronous=NORMAL`. The schema is idempotent and created on first start.

Table	Key columns	Purpose
meta	key, value	system_first_seen (start of learning)
apps	app_id, name, first_seen, last_seen	Every application ever seen on the network
flows	app_id, pid, proto, laddr:lport, raddr:rport, domain, ip_class, opened_at, closed_at, bytes_out, bytes_in, ja3n	One row per connection; indexed by (app_id, opened_at) and raddr
app_minutes	app_id, minute_ts, new_flows, unique_dests, new_dests, bytes_out, bytes_in, score, severity	Per-app per-minute history: feeds charts and sparklines
baselines (+ _dests, _ports, _ja3)	ewma_json, hour_hist, n_buckets; known sets	Learned behaviour per app; survives restarts
alerts	app_id, status, severity, peak_score, top_heuristic, summary, evidence_json	Alert history including all heuristic results at peak
policy_rules	scope (app | dest | app+dest), action (allow | deny | mute), expires_at	User decisions
user_actions	ts, alert_id, action, detail	Audit trail for false-positive statistics

9.1 Write strategy and retention

- The engine writes everything for a tick in **one transaction**. Typical cost: a few milliseconds.
- Flows are inserted when they open, byte counts are updated every 30 s, and the closing time is written when they close. A crash loses at most 30 s of byte counts.
- History views read only `app_minutes`, never the raw flows, so they stay fast.
- Readers (CLI commands) use their own connections; WAL means they never block the engine.
- Retention: closed flows 7 days (configurable), minute history 30 days; alerts, rules and baselines are kept.

10 Technical Implementation

10.1 Technology stack

Concern	Choice	Why
Language	Python 3.13	One language for capture, analysis and UI
Socket table, processes	psutil 7	Cross-platform, no admin needed for the socket table
Packet capture	Scapy 2.8 on Npcap	Mature dissectors for IP, TCP, UDP and DNS; BPF filtering
Configuration	pydantic 2 + PyYAML	Validated config; errors name the bad field
Domain parsing	tldextract (offline snapshot)	Correct eTLD+1 without network access
Command line	Typer + Rich	Typed options, readable tables
Dashboard	Textual 8	Full-screen terminal UI running on the same asyncio loop
Storage	SQLite (stdlib)	Zero setup; WAL for concurrent readers
Tests	pytest, pytest-asyncio, Hypothesis	Unit, integration and property-based tests
Lint	ruff	Fast lint and import sorting

10.2 Repository layout

```
mycafee/                          github.com/Nainikap/mycafee  (Python package: netsentry)
├─ README.md · pyproject.toml
├─ config/
│  ├─ rules.yaml                  weights, thresholds, profiles (Appendix A)
│  ├─ ports.yaml                  common and suspicious ports
│  ├─ lolbins.yaml                script hosts, suspicious parents, system binaries, staging paths
│  └─ blocklists/ ips.txt · domains.txt · ja3.txt
├─ src/netsentry/
│  ├─ cli.py · demo.py · config.py · models.py · clock.py · bus.py
│  ├─ capture/    socket_poller · process_resolver · sniffer · tls · idle · source
│  ├─ pipeline/   attributor · flow_table · enricher · engine
│  ├─ detection/  features · baseline · policy · scorer · alerts · heuristics/ (8 files)
│  ├─ storage/    schema.sql · db.py
│  └─ ui/tui/     app.py (Textual dashboard)
├─ tests/        unit/ (49 tests) · integration/ (5 tests) · factories.py
├─ tools/simulate/sim.py         harmless attack simulators
└─ docs/         01–07 design documents and this PDF
```

The source is about 3,800 lines of Python: capture 629, pipeline 674, detection 941, storage 187, UI 237, plus the CLI, models and configuration.

10.3 Core interfaces

```
class CaptureSource(Protocol):              # LiveSource · ReplaySource · ListSource
    full_mode: bool
    async def start(self) -> None
    async def sample(self) -> RawTick | None   # None = end of a replay
    async def stop(self) -> None

class Heuristic(Protocol):
    name: str; code: str                    # e.g. "frequency", "H2"
    def evaluate(self, win: AppWindow, base: Baseline | None,
                 ctx: HeuristicContext) -> HeuristicResult   # score 0..1 | None, evidence

def score(app_id, ts, results, policy: AppPolicy, cfg, dest_denied=None) -> Verdict
```

```
class Engine:
    async def run(self, max_ticks=None) -> None
    async def tick(self) -> TickResult | None      # one deterministic step (used by tests)
    def user_action(self, action, app_id, dest=None, minutes=None) -> str
```

10.4 Command-line interface

Command	Purpose
`netsentry run [--profile demo] [--ui tui|none] [--no-sniff] [--record F]`	Monitor this machine live
`netsentry demo [--speed 10] [--ui tui|none]`	Play the synthetic attack scenario through the real engine
`netsentry record --out F --minutes N`	Record raw capture ticks to JSONL
`netsentry replay F [--speed N]`	Replay a recording deterministically
`netsentry capture`	One snapshot of attributed connections (compare with netstat -ano)
`netsentry status / alerts [--json]`	Per-app summary and alert list from the database
`netsentry allow | deny | mute --app X [--dest D] [--minutes N]`	Add a policy rule
`netsentry rules [--delete ID]`	List or delete rules
`netsentry export --what alerts|flows --format csv|json`	Export data for the report
`netsentry doctor`	Check admin rights, Npcap, socket access, config and database

10.5 The dashboard

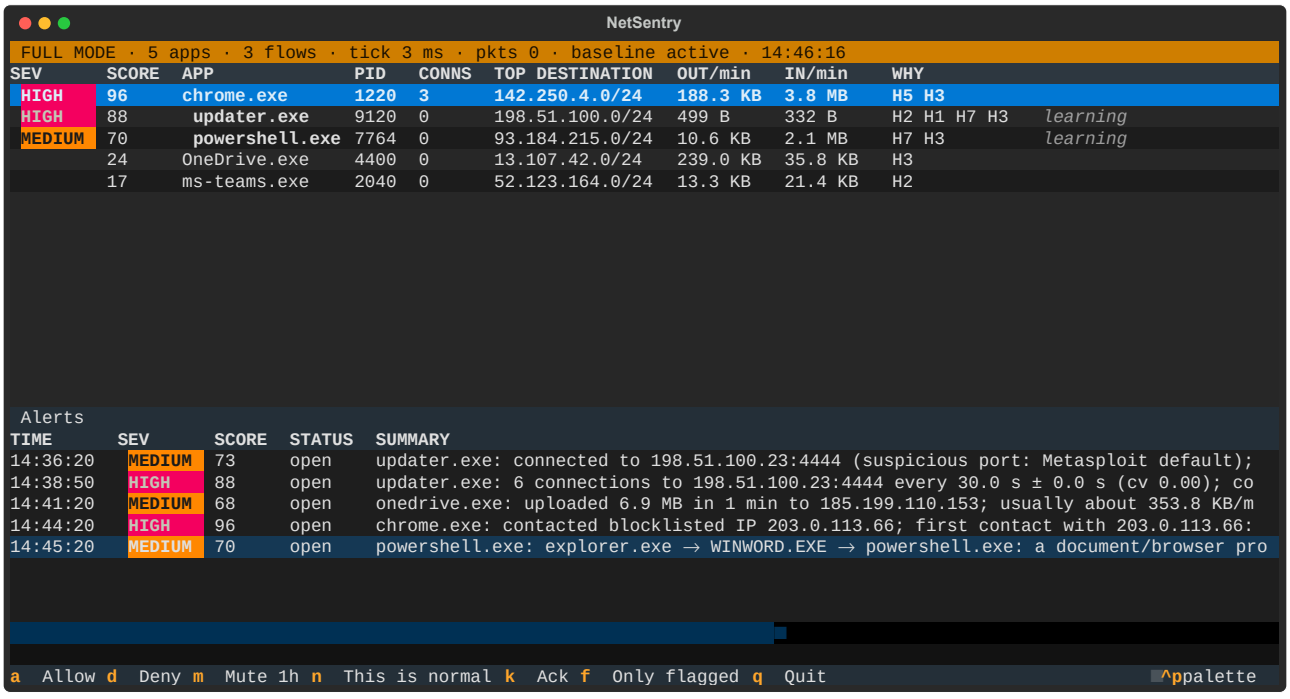


Figure 5. The Textual dashboard during `netsentry demo`: the live per-app table (severity, score, PID, connections, top destination, per-minute traffic, firing heuristics) above the alert list, with the key bar at the bottom.

The status line shows the capture mode, app and flow counts, tick time and the learning countdown. Selecting an app and pressing **Enter** opens a detail screen: parent chain, command line, baseline maturity, every heuristic with score × weight and evidence (n/a ones greyed out), open flows with bytes and JA3N, and one-hour sparklines of

connections and score.

Key	Action
Enter	Details for the selected app
a / d / m	Allow / deny / mute 1 h
n	“This is normal”: close the alert as a false positive and teach the baseline
k	Acknowledge
f	Show only flagged apps
q	Quit (flushes baselines and flows to the database)

10.6 Configuration

All tuning lives in plain text that is reloaded while NetSentry runs. An invalid edit to `rules.yaml` is logged and the previous configuration kept, so a typo cannot stop monitoring. Profiles override parts of the file; `--profile demo` shortens the learning period to 10 minutes. The full default file is in Appendix A.

10.7 Engineering conventions

- Detection code is pure: no I/O and no wall-clock calls. Time and data are passed in, which makes every heuristic unit-testable.
- Every evidence string is readable by a non-expert and contains the actual numbers.
- Config errors fail fast at startup with the field name (pydantic).
- Logs go to `%LOCALAPPDATA%\NetSentry\netsentry.log` with rotation; ticks over 250 ms are logged as warnings.
- Secrets, such as a future AbuseIPDB key, only ever come from environment variables.

11 Testing and Validation

11.1 Automated test suite

File	Tests	Covers
`tests/unit/test_heuristics.py`	24	Every sub-rule of H1–H8 at and around its boundary; allow-rule filtering; learning and n/a behaviour
`tests/unit/test_scorer.py`	12	All worked examples from Section 8.5; overrides; Hypothesis properties (score in 0–100, adding a signal never lowers it)
`tests/unit/test_flows.py`	7	Flow lifecycle, one-miss tolerance, byte attribution, UDP by port, pending → unknown, inbound detection, loopback
`tests/unit/test_capture_parsing.py`	6	ClientHello → SNI/JA3/JA3N, GREASE removal, JA3N stability under shuffling, truncation, DNS CNAME chains, IP helpers
`tests/integration/test_engine.py`	5	Synthetic scenario end to end in full and standard mode; quiet learning period; persistence; "this is normal"
`Total`	**54**	All passing (≈ 12 s)

11.2 Synthetic scenario results

`netsentry demo` generates 25 minutes of traffic (750 ticks) in the same format as a live recording. The first 12 minutes are normal Chrome browsing, a Microsoft Teams heartbeat every 60 s and light OneDrive sync. Four attacks follow. The chart shows the real scores the engine produced.

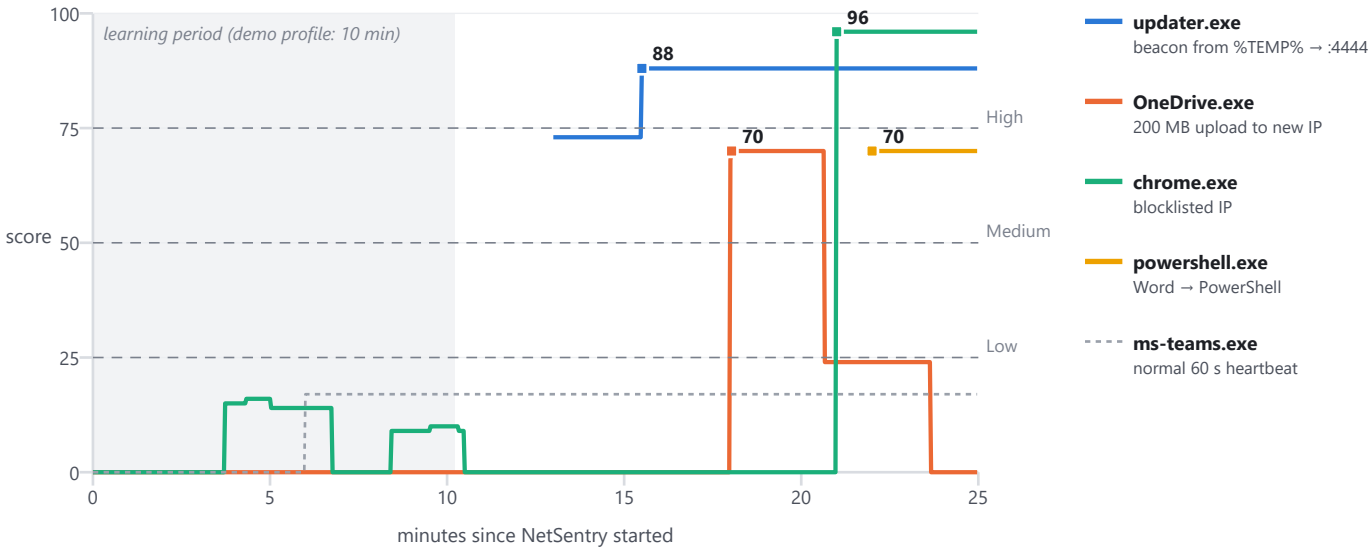


Figure 6. Per-app score over time in the synthetic scenario (demo profile). The dashed lines are the severity thresholds; the grey line is Teams' legitimate heartbeat, which stays below Low.

App	Attack	Severity	Peak	Top	Opened (min)
updater.exe	Beacon every 30 s from %TEMP% to :4444	High	88	H1	13.0
OneDrive.exe	200 MB upload to a new bare IP	Medium	70	H4	18.0
chrome.exe	Contact with a blocklisted IP	High	96	H5	21.0

App	Attack	Severity	Peak	Top	Opened (min)
powershell.exe	Word → PowerShell → new host	Medium	70	H7	22.0

All four attacks raised alerts, and nothing else did. Teams' perfectly regular 60-second heartbeat scored at most 17, because it goes to a destination the app already knows (beacon score × 0.3). In **standard mode** the same scenario still catches the beacon, the blocklist hit and the Word → PowerShell chain, but not the upload, because there are no byte counts. Both outcomes are asserted in the integration tests. The p95 engine tick on this scenario was 5.4 ms.

11.3 Attack simulators

`tools/simulate/sim.py` provides harmless simulators for live validation. They open ordinary TCP connections to hosts the tester chooses, and hold each one for a few seconds so the 2 s poll sees it. Nothing is installed or exfiltrated.

ID	Command	Simulates	Expected
S1	`sim.py beacon --host H --port 4444`	C2 beacon every 30 s ± 1 s	High (Medium while learning)
S2	`sim.py exfil --host H --mb 200`	Bulk upload	Medium/High (full mode)
S3	`sim.py oddport --port 31337`	Backdoor port	Low (45)
S4	`sim.py office`	WINWORD.EXE (renamed cmd) → PowerShell → internet	Medium
S5	`sim.py masquerade`	svchost.exe in %TEMP% (renamed curl)	Medium (60–67)
S6	`sim.py blocklist`	Adds a host to ips.txt, then connects	High (≥ 80)
—	`sim.py sink --ports 4444 9000`	Receiving end for S1/S2	

11.4 Live measurements

Check	Result (development laptop, standard mode)
Capture vs netstat	Every established TCP connection attributed to the correct process, with parent chains (e.g. msedgewebview2.exe ← ms-teams.exe ← explorer.exe)
Normal use, 75 s	13 apps and up to 80 flows tracked; no alerts
Tick cost	5–15 ms steady state; ~0.9 s once at startup (first process scan)
S1 beacon (12 s interval, local sink, learning period)	One Medium (53) alert: "connected to …:4444 (suspicious port: Metasploit default); 7 connections … every 11.7 s ± 1.4 s (cv 0.12) (damped while learning)"
Dashboard (headless Textual pilot)	Table and alerts render; Enter opens details; n applies "this is normal"

11.5 Bugs found by live testing

Running against a real machine found problems that synthetic tests had not. Each fix is now covered by a test.

Problem	Symptom	Fix
PID re-validation cost	Ticks of 700 ms–1.9 s	One parent-PID snapshot per batch; evict PIDs when the process exits instead of re-checking
Inbound connections looked outbound	A local server was flagged for "non-standard port 5xxxx" (the client's ephemeral port)	Flows accepted on a LISTEN port are marked inbound and skipped by H1/H2

Problem	Symptom	Fix
Beacon timing mixed endpoints	A real beacon was missed because unrelated connections to the same IP broke the regularity	Start times keyed by remote ip:port
Documentation IPs treated as LAN	Test IPs (203.0.113.x) got the LAN discount	Explicit list of LAN ranges
Two alerts per incident	The beacon opened one alert for H1, then another for H2	One active alert per app, updated in place
Stale evidence	Upload alert said 14.7 MB while 200 MB was being sent	Evidence refreshed at equal peak score
Windows console encoding	Crash printing → ± ≈ on cp1252 consoles	UTF-8 reconfigured at startup
Enter in the dashboard	Detail screen did not open (the table consumed the key)	Handle the table's row-selected event

11.6 Remaining validation work

- Run full mode from an administrator terminal: byte accuracy (V2: ±10 % on a known 100 MB transfer) and domain coverage (V3: ≥ 90 % of browser flows named).
- Run S2–S6 live and record true and false positives.
- Normal-day false-positive study: at least 6 hours after the 24 h learning period; alerts per hour, share marked “this is normal”, top causes, and the false-positive rate in hours 1–3 vs 4–6.
- Performance at load: replay a busy hour at 50× and confirm p95 tick under 250 ms.

12 Limitations

These are stated plainly because they affect how results should be read.

Limitation	Impact	Mitigation
Connections shorter than one poll (2 s)	Invisible in standard mode	Full mode sees their packets; unattributable ones go to “unknown”
QUIC / HTTP3 (UDP 443)	ClientHello is encrypted: no SNI or JA3	DNS answers still name the destination
DNS-over-HTTPS in browsers	DNS answers not visible	SNI and reverse DNS fill the gap
Scapy throughput	May drop packets above ~50 Mbit/s; byte counts approximate	Packet counter shown in the status line
Heuristic false positives	Backups, cloud sync, game launchers, app updates	Learning period, poisoning guard, feedback loop
Learning period	No “new destination” alerts for the first 24 h	Blocklist, lineage and port rules work from the first second
Single machine, Windows-focused	Idle detection and some lists are Windows-specific	Core is cross-platform via psutil
Metadata only	Cannot see what is being sent	By design: no decryption

13 Future Work

- **Web dashboard (P8):** FastAPI with REST + WebSocket on 127.0.0.1 with a startup token, plus a React front end with per-app history charts. It would subscribe to the existing EventBus.
- **Reputation lookups:** AbuseIPDB or VirusTotal free tier, cached, within the daily budget; confidence ≥ 75 → 0.9 inside H5.
- **Learned scorer:** an Isolation Forest trained on `app_minutes` behind the same Scorer interface, with the rule results kept as the explanation layer.
- **DNS-level monitoring:** detecting domain-generation-algorithm (DGA) patterns from query names.
- **JA4 fingerprints** and a JA3S server-side fingerprint.
- **Windows ETW** (Kernel-Network provider) for exact per-process byte counts without packet capture.
- **Optional enforcement:** "block with Windows Firewall" from an alert, with explicit confirmation.
- **System-tray service** and native desktop notifications for High alerts.

14 Running the Project

```
git clone https://github.com/Nainikap/mycafee && cd mycafee
python -m venv .venv
.venv\Scripts\pip install -e ".[dev]"

.venv\Scripts\netsentry doctor                # admin? Npcap? config valid?
.venv\Scripts\netsentry demo                  # synthetic attack scenario, no network needed
.venv\Scripts\netsentry run --profile demo    # live, 10-minute learning period
.venv\Scripts\python -m pytest                # 54 tests
```

For **full mode**, start the terminal with "Run as administrator" (Npcap must be installed). Without it, NetSentry runs in standard mode and says so in the status line. A presentation can use `netsentry demo` as a fallback if the venue network is unreliable.

14.1 Suggested demonstration (≈ 10 minutes)

- 1. `netsentry doctor`: show the capture mode and checks.
- 2. `netsentry run --profile demo`: the live table fills; open Chrome's detail screen (parent chain, destinations).
- 3. Run the beacon simulator: a Medium/High alert appears; read its evidence.
- 4. Run the exfiltration simulator (full mode), press **n**, and show the next run scoring lower.
- 5. Run the blocklist simulator: an immediate High.
- 6. Change a weight in `rules.yaml` and watch scores change within 10 s.

Appendix A: Default rules.yaml

```
# NetSentry rules — every weight and threshold the engine uses.
# Edit while NetSentry is running: the file is re-read within ~10 s.
# See docs/04-detection-engine.md for what each value means.
version: 1

engine:
  poll_interval_s: 2
  bucket_s: 60
  window_buckets: 5
  learning_period_h: 24
  min_buckets_mature: 30
  include_lan: true
  ignore_loopback: true
  retention_days: 7

baseline:
  ewma_alpha: 0.02
  min_std: 0.5
  z_lo: 3
  z_hi: 6

severity:
  low: 25
  medium: 50
  high: 75
  alert_min: medium
  resolve_after_s: 300
  dedupe_cooldown_s: 900

heuristics:
  unusual_port:    { enabled: true, weight: 0.45 }
  frequency:       { enabled: true, weight: 0.55, min_new_flows: 10, beacon_min_events: 6, cv_full: 0.10, cv_zero: 0.35, know
  new_destination: { enabled: true, weight: 0.35, direct_ip_bonus: 0.2, lan_factor: 0.5 }
  volume:          { enabled: true, weight: 0.60, min_bytes_out_per_min: 5000000, new_app_abs_bytes: 50000000 }
  blocklist:       { enabled: true, weight: 0.95, override_min_score: 80, allowed_cap: 60 }
  off_hours:       { enabled: true, weight: 0.25, idle_threshold_s: 600, quiet_hours: ["01:00", "06:00"], min_hour_share: 0.0
  lineage:         { enabled: true, weight: 0.60 }
  tls_fingerprint: { enabled: true, weight: 0.35 }

# Overrides applied by `--profile demo` so the whole story fits in a 15-minute demo.
profiles:
  demo:
    engine: { learning_period_h: 0.17, min_buckets_mature: 5 }
```

Appendix B: Glossary

Term	Meaning
Beacon	Regular, machine-timed connections from malware to its controller (C2) to ask for commands
C2	Command-and-control server used by an attacker to steer compromised machines
Coefficient of variation (cv)	Standard deviation divided by mean; near 0 means very regular timing
eTLD+1	Registrable domain: the public suffix plus one label (example.co.uk)
EWMA	Exponentially weighted moving average: a running mean that favours recent values
Exfiltration	Unauthorised transfer of data off a machine
5-tuple	Protocol, local IP, local port, remote IP, remote port: identifies one connection
GREASE	Random placeholder values browsers put in TLS handshakes to keep servers tolerant
JA3 / JA3N	MD5 fingerprint of a TLS ClientHello; JA3N sorts extensions so it stays stable
LOLBin	"Living off the land" binary: a built-in Windows tool (PowerShell, certutil…) abused by attackers
Noisy-OR	$S = 1 - \Pi(1 - p_i)$: the probability that at least one independent cause is active
Npcap	Windows packet capture driver used by Scapy and Wireshark
SNI	Server Name Indication: the host name a TLS client asks for, sent in clear text
TIME_WAIT	TCP state after a connection closes; on Windows it is reported under PID 0
z-score	How many standard deviations a value is from the mean

Appendix C: References

- GlassWire: Network Security Monitor & Firewall Tool, glasswire.com/features
- Objective Development: Little Snitch, obdev.at/littlesnitch
- The Zeek Project: About Zeek (Book of Zeek), docs.zeek.org/en/lts/about.html
- EC-Council: What is Wireshark?, eccouncil.org
- Open Information Security Foundation: Suricata, github.com/OISF/suricata
- Salesforce: JA3, a method for profiling SSL/TLS clients, github.com/salesforce/ja3
- abuse.ch: SSL Blacklist (JA3 fingerprints), sslbl.abuse.ch
- psutil documentation, psutil.readthedocs.io; Scapy documentation, scapy.readthedocs.io
- Microsoft: GetExtendedTcpTable and GetLastInputInfo (Win32 API documentation), learn.microsoft.com
- RFC 8446: TLS 1.3; RFC 8701: GREASE for TLS; RFC 1918: Private address space; RFC 5737: IPv4 documentation ranges
- Project source and design documents: github.com/Nainikap/mycafee (docs/01–07)